Lab 3: Register File

ECEN 127L: Tuesday 5:15 - 8:00 pm

Ryan Kiniris and Dylan Thornburg

Introduction

This lab looks into the basics of hierarchical, synchronous Verilog by designing and making a small register file. The register file will contain eight 8-bit registers, as well as two read ports and one write port that has a write-enable. Each register is assigned a 3-bit address, and the design contains bypassing capability so that if a read address matches the current write address while write-enable is declared, the write data appears immediately on the corresponding read output. We will then implement the file by first creating an 8-bit register with load-enable as a reusable building block, then instantiating eight of these registers and adding the necessary decode and bypass logic. We also added a reset feature for fun as we believe it makes the register file better and easier to use for testing setup.

Procedure

We began by finalizing the 8-bit register with load-enable from the pre-lab and confirming its interface. Next, we wrote the top-level register file module and instantiated eight copies of the register, declaring internal wires to carry each register's load-enable and data outputs. We then implemented logic that asserts exactly one load-enable based on the 3-bit write address when write_enable is high at the clock's rising edge. For the two read ports, we coded the bypass logic: if a read address equals the write address and write-enable is asserted in the same cycle, the read data is driven directly by the current write data; otherwise, it returns the stored data from the addressed register. Finally, we developed a testbench that generates a clock, sequences writes and reads to all addresses, exercises both read ports independently, and specifically tests the bypass case and normal read-after-write behavior. We simulated the design, inspected waveforms to verify timing at the rising clock edge, and iterated until both ports and the bypass path operated as expected.

Results

- What problems did you encounter while testing your steps yourself?
  No problems were encountered this time. We simply kept adding test cases until we demonstrated the full functionality of our register file.

- Did any problems arise when demonstrating for the TA? What were they? Explain your thoughts on how/why these test cases escaped your own testing.
  One error we had run into when demoing the lab to the TA was the back to back bypass, as at first, we did not include this function in our testbench. When including it, we initially saw the incorrect value appear for write_data (44 instead of 300). This of course was because our RF was 8-bits and 300 > 255. After correcting this and changing write_data to 100, it worked as expected. The TA also wanted us to change all our comments to print statements so it is easier to debug and understand expected behavior.

SIMULATION RESULTS

Register file test bench:

```
[dthornburg@linux402604 Lab3]$ ./simv -no_save
Chronologic VCS simulator copyright 1991-2023
Contains Synopsys proprietary information.
Compiler version V-2023.12-SP1-1_Full64; Runtime version V-2023.12-SP1-1_Full64;  Oct 14 17:34 2025
Start of the Register File Test


starting reset
write_addr=0, write_data =   0, read_addr1=0, read_addr0=0, read_data1=  x, read_data0=  x, reset=1, write_enable = 0, clk = 0, @      0
write_addr=0, write_data =   0, read_addr1=0, read_addr0=0, read_data1=  0, read_data0=  0, reset=1, write_enable = 0, clk = 1, @     10


turn off reset (idle)
write_addr=0, write_data =   0, read_addr1=0, read_addr0=0, read_data1=  0, read_data0=  0, reset=0, write_enable = 0, clk = 0, @     20
write_addr=0, write_data =   0, read_addr1=0, read_addr0=0, read_data1=  0, read_data0=  0, reset=0, write_enable = 0, clk = 1, @     30


basic write and read (write r6=128, then read r4)
write_addr=6, write_data = 128, read_addr1=4, read_addr0=0, read_data1=  0, read_data0=  0, reset=0, write_enable = 1, clk = 0, @     40
write_addr=6, write_data = 128, read_addr1=4, read_addr0=0, read_data1=  0, read_data0=  0, reset=0, write_enable = 1, clk = 1, @     50


read r6
write_addr=0, write_data =   0, read_addr1=6, read_addr0=6, read_data1=128, read_data0=128, reset=0, write_enable = 0, clk = 0, @     60
write_addr=0, write_data =   0, read_addr1=6, read_addr0=6, read_data1=128, read_data0=128, reset=0, write_enable = 0, clk = 1, @     70


write with write_enable=0 (shouldnt write)
write_addr=6, write_data = 255, read_addr1=6, read_addr0=6, read_data1=128, read_data0=128, reset=0, write_enable = 0, clk = 0, @     80
write_addr=6, write_data = 255, read_addr1=6, read_addr0=6, read_data1=128, read_data0=128, reset=0, write_enable = 0, clk = 1, @     90


new register write and read (write r3=80, then read r6)
write_addr=3, write_data =  80, read_addr1=0, read_addr0=6, read_data1=  0, read_data0=128, reset=0, write_enable = 1, clk = 0, @    100
write_addr=3, write_data =  80, read_addr1=0, read_addr0=6, read_data1=  0, read_data0=128, reset=0, write_enable = 1, clk = 1, @    110
write_addr=0, write_data =   0, read_addr1=3, read_addr0=3, read_data1= 80, read_data0= 80, reset=0, write_enable = 0, clk = 0, @    120
write_addr=0, write_data =   0, read_addr1=3, read_addr0=3, read_data1= 80, read_data0= 80, reset=0, write_enable = 0, clk = 1, @    130


bypass test on read_data0 (write r4 while reading r4)
write_addr=4, write_data =  33, read_addr1=3, read_addr0=4, read_data1= 80, read_data0= 33, reset=0, write_enable = 1, clk = 0, @    140
write_addr=4, write_data =  33, read_addr1=3, read_addr0=4, read_data1= 80, read_data0= 33, reset=0, write_enable = 1, clk = 1, @    150


bypass test on read_data1 (write r5 while reading r5)
write_addr=5, write_data =  77, read_addr1=5, read_addr0=4, read_data1= 77, read_data0= 33, reset=0, write_enable = 1, clk = 0, @    160
write_addr=5, write_data =  77, read_addr1=5, read_addr0=4, read_data1= 77, read_data0= 33, reset=0, write_enable = 1, clk = 1, @    170


bypass test on both ports (both read r2 while writing r2=200)
write_addr=2, write_data = 200, read_addr1=2, read_addr0=2, read_data1=200, read_data0=200, reset=0, write_enable = 1, clk = 0, @    180
write_addr=2, write_data = 200, read_addr1=2, read_addr0=2, read_data1=200, read_data0=200, reset=0, write_enable = 1, clk = 1, @    190


back to back bypass (both read r2 while writing r2=200
write_addr=2, write_data = 200, read_addr1=2, read_addr0=2, read_data1=200, read_data0=200, reset=0, write_enable = 1, clk = 0, @    200
write_addr=2, write_data = 200, read_addr1=2, read_addr0=2, read_data1=200, read_data0=200, reset=0, write_enable = 1, clk = 1, @    210
write_addr=2, write_data = 100, read_addr1=2, read_addr0=2, read_data1=100, read_data0=100, reset=0, write_enable = 1, clk = 0, @    220
write_addr=2, write_data = 100, read_addr1=2, read_addr0=2, read_data1=100, read_data0=100, reset=0, write_enable = 1, clk = 1, @    230


simultaneous read of two different registers w/out write
write_addr=0, write_data =   0, read_addr1=3, read_addr0=6, read_data1= 80, read_data0=128, reset=0, write_enable = 0, clk = 0, @    240
write_addr=0, write_data =   0, read_addr1=3, read_addr0=6, read_data1= 80, read_data0=128, reset=0, write_enable = 0, clk = 1, @    250
```

```
back to back writes to same register (r7=10 then r7=11)
write_addr=7, write_data =  10, read_addr1=7, read_addr0=7, read_data1= 10, read_data0= 10, reset=0, write_enable = 1, clk = 0, @          260
write_addr=7, write_data =  10, read_addr1=7, read_addr0=7, read_data1= 10, read_data0= 10, reset=0, write_enable = 1, clk = 1, @          270
write_addr=7, write_data =  11, read_addr1=7, read_addr0=7, read_data1= 11, read_data0= 11, reset=0, write_enable = 1, clk = 0, @          280
write_addr=7, write_data =  11, read_addr1=7, read_addr0=7, read_data1= 11, read_data0= 11, reset=0, write_enable = 1, clk = 1, @          290


confirm r7 holds 11
write_addr=0, write_data =   0, read_addr1=7, read_addr0=7, read_data1= 11, read_data0= 11, reset=0, write_enable = 0, clk = 0, @          300
write_addr=0, write_data =   0, read_addr1=7, read_addr0=7, read_data1= 11, read_data0= 11, reset=0, write_enable = 0, clk = 1, @          310


back to back writes to different registers (r0=44 then r1=55)
write_addr=0, write_data =  44, read_addr1=0, read_addr0=0, read_data1= 44, read_data0= 44, reset=0, write_enable = 1, clk = 0, @          320
write_addr=0, write_data =  44, read_addr1=0, read_addr0=0, read_data1= 44, read_data0= 44, reset=0, write_enable = 1, clk = 1, @          330
write_addr=1, write_data =  55, read_addr1=1, read_addr0=1, read_data1= 55, read_data0= 55, reset=0, write_enable = 1, clk = 0, @          340
write_addr=1, write_data =  55, read_addr1=1, read_addr0=1, read_data1= 55, read_data0= 55, reset=0, write_enable = 1, clk = 1, @          350


confirm r0=44 and r1=55 (no write)
write_addr=0, write_data =   0, read_addr1=0, read_addr0=1, read_data1= 44, read_data0= 55, reset=0, write_enable = 0, clk = 0, @          360
write_addr=0, write_data =   0, read_addr1=0, read_addr0=1, read_data1= 44, read_data0= 55, reset=0, write_enable = 0, clk = 1, @          370


boundary test (read R0 and R7)
write_addr=0, write_data =   0, read_addr1=0, read_addr0=7, read_data1= 44, read_data0= 11, reset=0, write_enable = 0, clk = 0, @          380
write_addr=0, write_data =   0, read_addr1=0, read_addr0=7, read_data1= 44, read_data0= 11, reset=0, write_enable = 0, clk = 1, @          390


reset
write_addr=0, write_data =   0, read_addr1=0, read_addr0=0, read_data1= 44, read_data0= 44, reset=1, write_enable = 0, clk = 0, @          400
write_addr=0, write_data =   0, read_addr1=0, read_addr0=0, read_data1=  0, read_data0=  0, reset=1, write_enable = 0, clk = 1, @          410


post-reset reads should be 0
write_addr=0, write_data =   0, read_addr1=1, read_addr0=0, read_data1=  0, read_data0=  0, reset=0, write_enable = 0, clk = 0, @          420
write_addr=0, write_data =   0, read_addr1=1, read_addr0=0, read_data1=  0, read_data0=  0, reset=0, write_enable = 0, clk = 1, @          430
write_addr=0, write_data =   0, read_addr1=3, read_addr0=2, read_data1=  0, read_data0=  0, reset=0, write_enable = 0, clk = 0, @          440
write_addr=0, write_data =   0, read_addr1=3, read_addr0=2, read_data1=  0, read_data0=  0, reset=0, write_enable = 0, clk = 1, @          450
write_addr=0, write_data =   0, read_addr1=5, read_addr0=4, read_data1=  0, read_data0=  0, reset=0, write_enable = 0, clk = 0, @          460
write_addr=0, write_data =   0, read_addr1=5, read_addr0=4, read_data1=  0, read_data0=  0, reset=0, write_enable = 0, clk = 1, @          470
write_addr=0, write_data =   0, read_addr1=7, read_addr0=6, read_data1=  0, read_data0=  0, reset=0, write_enable = 0, clk = 0, @          480
write_addr=0, write_data =   0, read_addr1=7, read_addr0=6, read_data1=  0, read_data0=  0, reset=0, write_enable = 0, clk = 1, @          490


End of the Register File Test
$finish called from file "tb.v", line 170.
$finish at simulation time                 500
           V C S   S i m u l a t i o n   R e p o r t
Time: 500
CPU Time:      0.200 seconds;       Data structure size:   0.0Mb
Tue Oct 14 17:34:15 2025
[dthornburg@linux402604 Lab3]$
```

Conclusion

Overall, in this lab we created, designed, and implemented our register file of eight eight-bit registers. We tested all functions of the register file and it passed every single one while implementing an extra reset function. Our register file could confidently be used on a small chip where power needs to be extremely low and the bitstream doesn't have to be very large.

Appendix

CODE

```
module eightBitReg (
	input [7:0] D,
	input clk,
	input en,
	input rst, //not required, but makes life better
	output reg [7:0] Q
);
	always @(posedge clk) //because Wolfe said rising only
		if (rst) begin
			Q <= 8'b0;
		end else if (en) begin
			Q <= D;
		end
endmodule
```

```
module eightBitRegFile (
	input clk,
	input reset, // not required
	input write_enable,
	input [2:0] write_addr,
	input [7:0] write_data,
	input [2:0] read_addr1,
	input [2:0] read_addr0,
	output [7:0] read_data1,
	output [7:0] read_data0
);
	wire [7:0] en;
	wire [7:0] Q [7:0];
	//despite what you may think, a case statement is actually more lines of code
	assign en[0] = write_enable && (write_addr == 3'd0);
	assign en[1] = write_enable && (write_addr == 3'd1);
	assign en[2] = write_enable && (write_addr == 3'd2);
	assign en[3] = write_enable && (write_addr == 3'd3);
	assign en[4] = write_enable && (write_addr == 3'd4);
	assign en[5] = write_enable && (write_addr == 3'd5);
	assign en[6] = write_enable && (write_addr == 3'd6);
	assign en[7] = write_enable && (write_addr == 3'd7);
	// added reset to make analyzing easier
	eightBitReg zero(write_data, clk, en[0], reset, Q[0]);
	eightBitReg one(write_data, clk, en[1], reset, Q[1]);
	eightBitReg two(write_data, clk, en[2], reset, Q[2]);
```

```
	eightBitReg three(write_data, clk, en[3], reset, Q[3]);
	eightBitReg four(write_data, clk, en[4], reset, Q[4]);
	eightBitReg five(write_data, clk, en[5], reset, Q[5]);
	eightBitReg six(write_data, clk, en[6], reset, Q[6]);
	eightBitReg seven(write_data, clk, en[7], reset, Q[7]);
	//Bypass. If read addr == write addr and write is asserted, write data is read
	assign read_data0 = write_enable && (write_addr == read_addr0) ? write_data :
Q[read_addr0];
	assign read_data1 = write_enable && (write_addr == read_addr1) ? write_data :
Q[read_addr1];
endmodule
```

```
module testbench();

reg [2:0] write_addr, read_addr1, read_addr0;
reg [7:0] write_data;
reg clk, reset, write_enable;
wire [7:0] read_data1, read_data0;

eightBitRegFile dut(clk, reset, write_enable, write_addr, write_data, read_addr1,
read_addr0, read_data1, read_data0);

initial begin
	$display("Start of the Register File Test");
	$monitor("write_addr=%d, write_data = %d, read_addr1=%d, read_addr0=%d,
read_data1=%d, read_data0=%d, reset=%d, write_enable = %d, clk = %d, @ %d",
write_addr, write_data, read_addr1, read_addr0, read_data1, read_data0, reset,
write_enable, clk, $time);
$display("\n");

$display("starting reset");
write_addr=3'd0; write_data=8'd0; read_addr1=3'd0; read_addr0=3'd0; reset=1'd1;
write_enable=1'd0; clk=1'd0;
#10
write_addr=3'd0; write_data=8'd0; read_addr1=3'd0; read_addr0=3'd0; reset=1'd1;
write_enable=1'd0; clk=1'd1;
#10
$display("\n");

$display("turn off reset (idle)");
write_addr=3'd0; write_data=8'd0; read_addr1=3'd0; read_addr0=3'd0; reset=1'd0;
write_enable=1'd0; clk=1'd0;
#10
write_addr=3'd0; write_data=8'd0; read_addr1=3'd0; read_addr0=3'd0; reset=1'd0;
write_enable=1'd0; clk=1'd1;
#10
$display("\n");

$display("basic write and read (write r6=128, then read r4)");
```

```
write_addr=3'd6; write_data=8'd128; read_addr1=3'd4; read_addr0=3'd0; reset=1'd0;
write_enable=1'd1; clk=1'd0;
#10
write_addr=3'd6; write_data=8'd128; read_addr1=3'd4; read_addr0=3'd0; reset=1'd0;
write_enable=1'd1; clk=1'd1;
#10
$display("\n");

$display("read r6");
write_addr=3'd0; write_data=8'd0; read_addr1=3'd6; read_addr0=3'd6; reset=1'd0;
write_enable=1'd0; clk=1'd0;
#10
write_addr=3'd0; write_data=8'd0; read_addr1=3'd6; read_addr0=3'd6; reset=1'd0;
write_enable=1'd0; clk=1'd1;
#10
$display("\n");

$display("write with write_enable=0 (shouldnt write)");
write_addr=3'd6; write_data=8'd255; read_addr1=3'd6; read_addr0=3'd6; reset=1'd0;
write_enable=1'd0; clk=1'd0;
#10
write_addr=3'd6; write_data=8'd255; read_addr1=3'd6; read_addr0=3'd6; reset=1'd0;
write_enable=1'd0; clk=1'd1;
#10
$display("\n");

$display("new register write and read (write r3=80, then read r6)");
write_addr=3'd3; write_data=8'd80; read_addr1=3'd0; read_addr0=3'd6; reset=1'd0;
write_enable=1'd1; clk=1'd0;
#10
write_addr=3'd3; write_data=8'd80; read_addr1=3'd0; read_addr0=3'd6; reset=1'd0;
write_enable=1'd1; clk=1'd1;
#10
write_addr=3'd0; write_data=8'd0; read_addr1=3'd3; read_addr0=3'd3; reset=1'd0;
write_enable=1'd0; clk=1'd0;
#10
write_addr=3'd0; write_data=8'd0; read_addr1=3'd3; read_addr0=3'd3; reset=1'd0;
write_enable=1'd0; clk=1'd1;
#10
$display("\n");

$display("bypass test on read_data0 (write r4 while reading r4)");
write_addr=3'd4; write_data=8'd33; read_addr1=3'd3; read_addr0=3'd4; reset=1'd0;
write_enable=1'd1; clk=1'd0;
#10
write_addr=3'd4; write_data=8'd33; read_addr1=3'd3; read_addr0=3'd4; reset=1'd0;
write_enable=1'd1; clk=1'd1;
#10
$display("\n");
```

```
$display("bypass test on read_data1 (write r5 while reading r5)");
write_addr=3'd5; write_data=8'd77; read_addr1=3'd5; read_addr0=3'd4; reset=1'd0;
write_enable=1'd1; clk=1'd0;
#10
write_addr=3'd5; write_data=8'd77; read_addr1=3'd5; read_addr0=3'd4; reset=1'd0;
write_enable=1'd1; clk=1'd1;
#10
$display("\n");

$display("bypass test on both ports (both read r2 while writing r2=200)");
write_addr=3'd2; write_data=8'd200; read_addr1=3'd2; read_addr0=3'd2; reset=1'd0;
write_enable=1'd1; clk=1'd0;
#10
write_addr=3'd2; write_data=8'd200; read_addr1=3'd2; read_addr0=3'd2; reset=1'd0;
write_enable=1'd1; clk=1'd1;
#10
$display("\n");

$display("back to back bypass (both read r2 while writing r2=200");
write_addr=3'd2; write_data=8'd200; read_addr1=3'd2; read_addr0=3'd2; reset=1'd0;
write_enable=1'd1; clk=1'd0;
#10
write_addr=3'd2; write_data=8'd200; read_addr1=3'd2; read_addr0=3'd2; reset=1'd0;
write_enable=1'd1; clk=1'd1;
#10
write_addr=3'd2; write_data=8'd100; read_addr1=3'd2; read_addr0=3'd2; reset=1'd0;
write_enable=1'd1; clk=1'd0;
#10
write_addr=3'd2; write_data=8'd100; read_addr1=3'd2; read_addr0=3'd2; reset=1'd0;
write_enable=1'd1; clk=1'd1;
#10
$display("\n");

$display("simultaneous read of two different registers w/out write");
write_addr=3'd0; write_data=8'd0; read_addr1=3'd3; read_addr0=3'd6; reset=1'd0;
write_enable=1'd0; clk=1'd0;
#10
write_addr=3'd0; write_data=8'd0; read_addr1=3'd3; read_addr0=3'd6; reset=1'd0;
write_enable=1'd0; clk=1'd1;
#10
$display("\n");

$display("back to back writes to same register (r7=10 then r7=11)");
write_addr=3'd7; write_data=8'd10; read_addr1=3'd7; read_addr0=3'd7; reset=1'd0;
write_enable=1'd1; clk=1'd0;
#10
write_addr=3'd7; write_data=8'd10; read_addr1=3'd7; read_addr0=3'd7; reset=1'd0;
write_enable=1'd1; clk=1'd1;
#10
```

```
write_addr=3'd7; write_data=8'd11; read_addr1=3'd7; read_addr0=3'd7; reset=1'd0;
write_enable=1'd1; clk=1'd0;
#10
write_addr=3'd7; write_data=8'd11; read_addr1=3'd7; read_addr0=3'd7; reset=1'd0;
write_enable=1'd1; clk=1'd1;
#10
$display("\n");

$display("confirm r7 holds 11");
write_addr=3'd0; write_data=8'd0; read_addr1=3'd7; read_addr0=3'd7; reset=1'd0;
write_enable=1'd0; clk=1'd0;
#10
write_addr=3'd0; write_data=8'd0; read_addr1=3'd7; read_addr0=3'd7; reset=1'd0;
write_enable=1'd0; clk=1'd1;
#10
$display("\n");

$display("back to back writes to different registers (r0=44 then r1=55)");
write_addr=3'd0; write_data=8'd44; read_addr1=3'd0; read_addr0=3'd0; reset=1'd0;
write_enable=1'd1; clk=1'd0;
#10
write_addr=3'd0; write_data=8'd44; read_addr1=3'd0; read_addr0=3'd0; reset=1'd0;
write_enable=1'd1; clk=1'd1;
#10
write_addr=3'd1; write_data=8'd55; read_addr1=3'd1; read_addr0=3'd1; reset=1'd0;
write_enable=1'd1; clk=1'd0;
#10
write_addr=3'd1; write_data=8'd55; read_addr1=3'd1; read_addr0=3'd1; reset=1'd0;
write_enable=1'd1; clk=1'd1;
#10
$display("\n");

$display("confirm r0=44 and r1=55 (no write)");
write_addr=3'd0; write_data=8'd0; read_addr1=3'd0; read_addr0=3'd1; reset=1'd0;
write_enable=1'd0; clk=1'd0;
#10
write_addr=3'd0; write_data=8'd0; read_addr1=3'd0; read_addr0=3'd1; reset=1'd0;
write_enable=1'd0; clk=1'd1;
#10
$display("\n");

$display("boundary test (read R0 and R7)");
write_addr=3'd0; write_data=8'd0; read_addr1=3'd0; read_addr0=3'd7; reset=1'd0;
write_enable=1'd0; clk=1'd0;
#10
write_addr=3'd0; write_data=8'd0; read_addr1=3'd0; read_addr0=3'd7; reset=1'd0;
write_enable=1'd0; clk=1'd1;
#10
$display("\n");
```

```
$display("reset");
write_addr=3'd0; write_data=8'd0; read_addr1=3'd0; read_addr0=3'd0; reset=1'd1;
write_enable=1'd0; clk=1'd0;
#10
write_addr=3'd0; write_data=8'd0; read_addr1=3'd0; read_addr0=3'd0; reset=1'd1;
write_enable=1'd0; clk=1'd1;
#10
$display("\n");

$display("post-reset reads should be 0");
write_addr=3'd0; write_data=8'd0; read_addr1=3'd1; read_addr0=3'd0; reset=1'd0;
write_enable=1'd0; clk=1'd0;
#10
write_addr=3'd0; write_data=8'd0; read_addr1=3'd1; read_addr0=3'd0; reset=1'd0;
write_enable=1'd0; clk=1'd1;
#10
write_addr=3'd0; write_data=8'd0; read_addr1=3'd3; read_addr0=3'd2; reset=1'd0;
write_enable=1'd0; clk=1'd0;
#10
write_addr=3'd0; write_data=8'd0; read_addr1=3'd3; read_addr0=3'd2; reset=1'd0;
write_enable=1'd0; clk=1'd1;
#10
write_addr=3'd0; write_data=8'd0; read_addr1=3'd5; read_addr0=3'd4; reset=1'd0;
write_enable=1'd0; clk=1'd0;
#10
write_addr=3'd0; write_data=8'd0; read_addr1=3'd5; read_addr0=3'd4; reset=1'd0;
write_enable=1'd0; clk=1'd1;
#10
write_addr=3'd0; write_data=8'd0; read_addr1=3'd7; read_addr0=3'd6; reset=1'd0;
write_enable=1'd0; clk=1'd0;
#10
write_addr=3'd0; write_data=8'd0; read_addr1=3'd7; read_addr0=3'd6; reset=1'd0;
write_enable=1'd0; clk=1'd1;
#10
$display("\n");

$display("End of the Register File Test");
$finish;

end

endmodule
```